

EVEGLYPHDESIGN · HAWKINS TWIN PLATFORM

Warranty GENE — Luke Live Brief

EgD-WGB-007 · Proving Luke's DMS logins pull real data · One hour to demo

Purpose. In the next hour, we prove that Luke's own CDK Drive and Lightspeed EVO logins pull real warranty data for one Peterbilt VIN and one BRP VIN, driven headlessly from Luke's repo admin credentials. No Fortellis subscription. No Client Portal API subscription. No Azure yet. Everything runs on GitHub Actions in the [hawkins-twin-platform](#) repo. Tobias moves it to Azure after we prove it works.

The three-step demo

1. **Luke opens the intake page**, fills in his CDK Drive and Lightspeed EVO logins, clicks *Encrypt in browser*, copies the encrypted blob. Intake page: [Warranty GENE — Credential intake](#)
2. **Store the blob**: dispatch the [Store credentials workflow](#) with the blob pasted into the `payload` input. Writes `warranty-gene/dms-pull/credentials.age` to the repo.
3. **Pull a VIN**: dispatch the [Pull VIN workflow](#) with `side=cdk` and a Peterbilt VIN, then again with `side=lightspeed` and a BRP VIN. Result JSON commits to `warranty-gene/dms-pull/pulls/`.

What Dany needs to do BEFORE Luke arrives (10 min)

One-time `age` key setup — this is what makes the credential file readable by the workflow and by nothing else.

1. On any machine with `age` installed (`brew install age` on macOS), run:

```
age-keygen -o warranty-gene-age.key
```

This prints the public key (`age1...`) to stderr and writes the private key into the file (`AGE-SECRET-KEY-...`).

1. Open the repo secrets page: [Actions secrets — hawkins-twin-platform](#). Add two:
 - `AGE_PUBLIC_KEY` — the public key string, one line, starts `age1` .
 - `AGE_PRIVATE_KEY` — the full contents of `warranty-gene-age.key` , starts `AGE-SECRET-KEY-` .
1. Delete `warranty-gene-age.key` from the local machine. GitHub Actions has both halves. Nothing else needs it.

That is the whole setup. If the **Store credentials workflow** fails at the *Verify AGE_PUBLIC_KEY* step, the secrets were not saved — redo step 2.

What Luke needs when he arrives

Have these ready — the intake form will not submit without them. If Luke does not have them at hand, we ask him for them one at a time as he types.

Field	What to bring	Why it matters
CDK login URL	The exact URL Luke opens to sign into CDK Drive (the dealer-scoped one, not <code>cdk.com</code> marketing)	The reader navigates to that URL first; wrong URL = login screen never appears
CDK username + password	The credentials Luke uses today	The reader logs in as Luke
CDK MFA method	TOTP (authenticator app), SMS, push, or SSO	Determines whether the reader can complete login unattended
CDK TOTP seed — only if MFA=TOTP	The base32 secret from the authenticator setup screen (find it via "can't scan QR" in the app)	Without the seed the reader cannot generate the 6-digit code
Lightspeed login URL	The exact URL Luke opens for Lightspeed EVO — Torque Motorsports scoped	Same
Lightspeed username + password	Same	
Lightspeed MFA method	TOTP, SMS, push	Same
Lightspeed TOTP seed — only if MFA=TOTP	Same	
One Peterbilt VIN	A VIN in CDK Drive today, ideally one where Luke already knows the warranty answer	Lets us eyeball whether the pull is right
One BRP VIN	A unit in Lightspeed EVO	Same

MFA reality check

The reader handles four MFA shapes. Only one of them works fully unattended.

- **None** — works.
- **TOTP (authenticator app)** — works if Luke gives us the base32 seed. This is the target. If MFA is currently on but not TOTP, Luke can switch it to TOTP in both DMSes in about 60 seconds each — the account settings page has an authenticator-app option.

-
- **SMS or push** — the reader will fail with a clear error. Luke has to switch to TOTP on that account, or approve each pull as it happens (which defeats the point).
 - **SSO through Peterbilt / PACCAR** — the reader fails, because it would need the IdP credentials, not the CDK ones. If CDK is SSO-federated at Peterbilt Atlantic, we have to capture the Peterbilt SSO credentials instead — Luke will know from his own experience whether "log into CDK" today means typing a CDK password or clicking a Peterbilt login button.

The single fastest thing Luke can do on his own before the call: **turn on TOTP** on both CDK Drive and Lightspeed EVO if it isn't already, using any authenticator app (1Password, Google Authenticator, Authy, Microsoft Authenticator all work). Save the setup seed as text somewhere he can copy from, not just scan.

What the first pull will look like

The reader is deliberately dumb on the first pass. It logs in, finds the VIN search box, types the VIN, hits Enter, then captures the entire text of the result page into a single field in the JSON. That gives us proof of principle — real login, real DMS, real VIN, real data — and a full copy of what the DMS shows so we can write targeted extractors ("base warranty start date", "extended plans", "open ROs") without another authentication.

The pull output file lands at `warranty-gene/dms-pull/pulls/{side}-{vin}-{timestamp}.json` with:

- `success: true` and an `extracted.raw.main_text` field containing the DMS VIN screen as plain text, or
- `success: false` with a Playwright trace attached to the workflow run as an artifact — we open the trace in a browser and see exactly what the reader saw, screenshot by screenshot.

If the first pull fails, it will be because the selectors in `readers/cdk.py` or `readers/lightspeed.py` don't match Luke's specific dealer's screen layout. That is a code change we make in the repo, commit, and re-dispatch — Luke does not re-enter credentials.

Expected timeline (from Luke's first click)

- Minute 0–3: Luke fills the intake form and copies the blob.
- Minute 3–5: dispatch the store-credentials workflow, watch it turn green.
- Minute 5–15: first pull attempt for CDK. Likely one or two selector fixes on our side, each a commit.
- Minute 15–25: first pull attempt for Lightspeed. Same.
- Minute 25–35: both sides pulling clean raw-text output. That is the demo.

After the demo

- Write targeted extractors — turn the raw-text dump into structured `warranty.base` , `warranty.extended` , `work_orders_open` fields. Half a day of work looking at real DMS screens.
- Wire the reader output into the blueprint's step-2 engine (Uplift / Rank / Warranty scan).
- Hand the whole thing to Tobias with the [DMS pull README](#), which spells out what changes when it moves to Azure: swap the age file read for an Azure Key Vault call, swap the git commit for an Azure Blob write, keep the reader modules unchanged.

Working surfaces

- [Intake page](#)
- [Store credentials workflow](#)
- [Pull VIN workflow](#)
- [DMS pull README](#)
- [Pulls directory](#)
- [Blueprint v2 \(EgD-WGB-006\)](#)

© 2026 EVEglyphDesign. All rights reserved. Controlled copy.

Key ID `EgD-KEY-2026-07` .

Pour le bien-être du peuple.